

App Dev Project Report

1. Student Details

Name: Sarthak Chaudhary

Roll Number: DS23f1000304

Email: 23f1000304@ds.study.iitm.ac.in

About Me: I'm a BS in Data Science student at IITM and I'm also into music where I write, sing and produce my independent music.

2. Project Details

Project Title: Hospital Management System

Problem Statement:

To develop a Placement Portal Application that connects students and recruiters. Companies can post job drives, students can apply and track status, and an admin manages approvals and oversees the entire placement workflow.

Approach:

The system was built starting with a scalable and well-structured database. The backend was developed using Flask to create secure APIs for authentication, job postings, and application tracking. The frontend was built using Vue.js with a modular design that supports multiple screen sizes. The platform ensures smooth interaction between students, recruiters, and the admin.

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)/Gemini Pro** minimally to assist with **JavaScript or AJAX snippets (2%)**, **chart or data visualization (2%)**, and **testing or debugging (1%)**. ***The total AI involvement is around 5%***, limited to small code suggestions and formatting. ***All core implementation, logic, debugging, and integration were done manually.***

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for database operations
Jinja2	Rendering dynamic HTML templates
Vue.js	Frontend development and UI interaction
Bootstrap 5	Responsive design and styling
Chart.js	Data visualization such as activity trends
Redis	Caching and message brokering
Celery	Background jobs like reminders and report generation
JSON Web Token	Secure authentication and API protection
SQLite	Database for data storage

5. Database Schema / ER Diagram

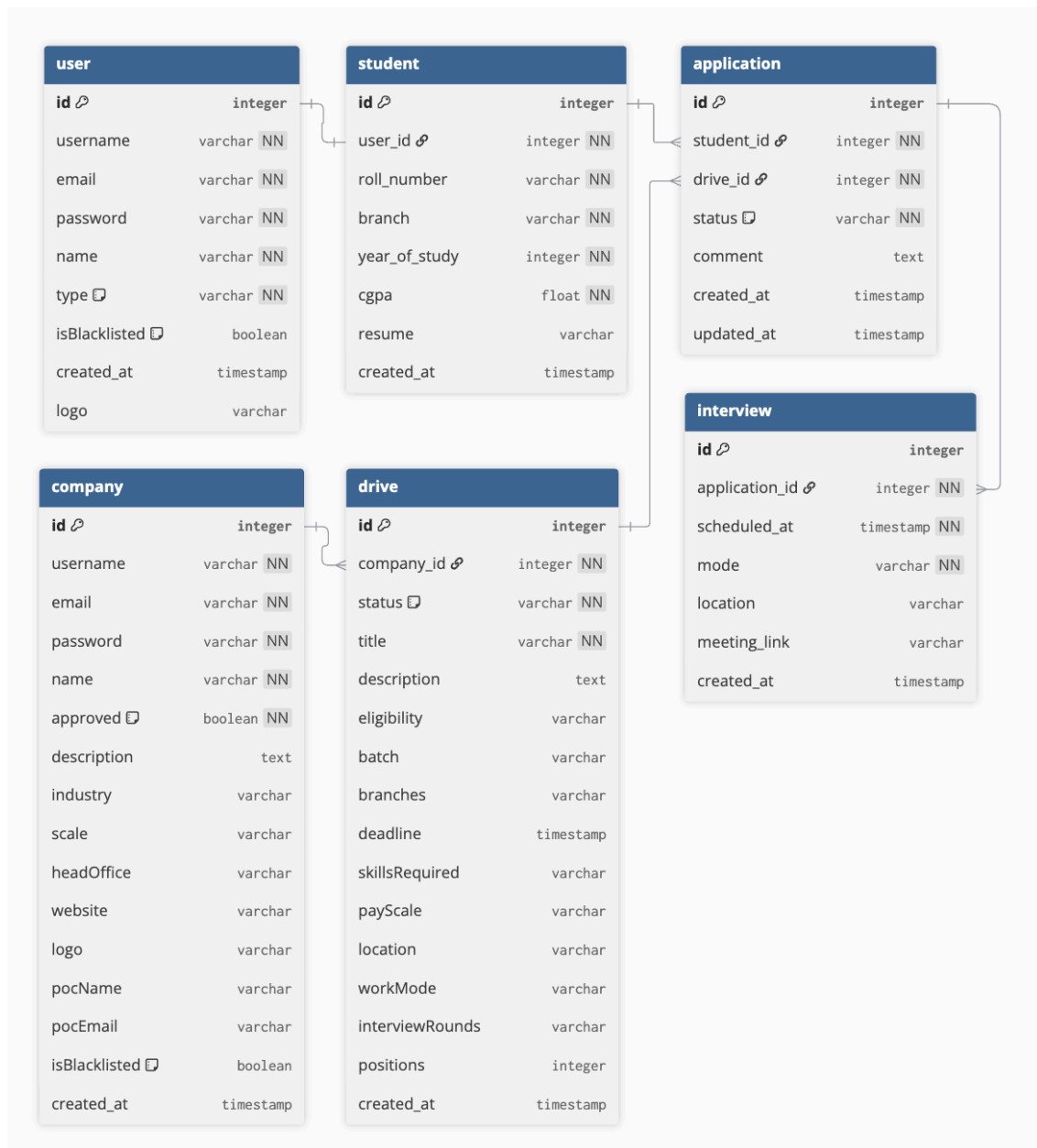
Tables:

1. **User** — stores basic account details (id, username, email, password, name, type, isBlacklisted, created_at, logo)
2. **Student** — stores student-specific details (id, user_id, roll_number, branch, year_of_study, cgpa, resume, created_at)
3. **Company** — stores recruiter/company profiles (id, username, email, password, name, approved, description, industry, scale, headOffice, website, logo, pocName, pocEmail, isBlacklisted, created_at)
4. **Drive** — stores job drives or placement opportunities (id, company_id, status, title, description, eligibility, batch, branches, deadline, skillsRequired, payScale, location, workMode, interviewRounds, positions, created_at)
5. **Application** — stores student applications for drives (id, student_id, drive_id, status, comment, created_at, updated_at)

6. **Interview** — stores interview scheduling details (id, application_id, scheduled_at, mode, location, meeting_link, created_at)

Relationships:

- **One-to-One** → User → Student
- **One-to-Many** → Company → Drive
- **One-to-Many** → Student → Application
- **One-to-Many** → Drive → Application
- **One-to-Many** → Application → Interview



6. API Resource Endpoints (Major)

Endpoint	Method	Description
/api/login	POST	Authenticate user and return JWT token
/api/me	GET	Get current authenticated user details
/api/student-register	POST	Register a new student account
/api/company-register	POST	Register a new company (pending approval)
/api/admin/companies	GET	Fetch all companies for admin dashboard (with caching)
/api/admin/all-students	GET	Fetch all students for admin
/api/admin/approve_company/{id}	PUT	Approve a company
/api/admin/approve_drive/{id}	PUT	Approve a job drive
/api/company/create-drive	POST	Create a new job drive
/api/company/update-drive/{id}	PUT	Update an existing drive
/api/company/drive-applications/{id}	GET	Get applications for a company drive
/api/company/update-application-status/{id}	PUT	Update status of a student application
/api/company/schedule-interview/{id}	POST	Schedule interview and trigger email (Celery)
/api/student/active-drives	GET	Fetch all active job drives
/api/student/apply/{id}	POST	Apply to a specific drive
/api/student/applications	GET	Get all applications of a student
/api/student/interviews	GET	Fetch scheduled interviews for student
/api/student/profile	GET	Get student profile
/api/student/profile	POST	Create or update student profile

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – Main entry point of the Flask application, initializes routes, configurations, and extensions
- **application/models.py** – Defines database schema using SQLAlchemy
- **application/controllers/** – Contains modular route handlers (admin.py, auth.py, company.py, student.py, system.py) managing different roles and functionalities
- **application/database.py** – Handles database connection and initialization
- **application/extensions.py** – Configures external services like JWT and Redis
- **application/tasks.py** – Background jobs using Celery for emails, reminders, and CSV export
- **application/utils.py** – Utility functions including caching and helper methods
- **instance/config.py** – Application configuration and environment settings
- **celery_worker.py** – Worker process for handling asynchronous tasks
- **frontend/** – Built using Vue.js with a component-based and responsive structure

Implemented Features:

- User authentication with role-based access (Admin, Student, Recruiter)
- Student and company registration with approval workflow
- Blacklist and whitelist system for students and companies
- Job drive creation and management by recruiters
- Students can explore drives, apply, and track application status
- Admin can manage companies, drives, students, and applications
- Interview scheduling and tracking system
- Background jobs for reminders, emails, and CSV export using Celery and Redis
- Dashboard analytics with data visualization using Chart.js
- Search functionality across students, companies, and drives
- Secure authentication using JSON Web Token
- Caching for performance optimization using Redis
- Responsive frontend built with Vue.js
- SQLAlchemy ORM with well-defined relationships
- Error handling and input validation across the system

Additional Features:

- Real-time status updates for applications and interview stages

- Email notifications for interview scheduling and reminders using Celery
 - CSV export of application and placement data for recruiters and students
 - Advanced filtering and search for drives, students, and companies
 - Caching of frequently accessed data using Redis to improve performance
 - Role-based dashboard views with personalized data insights
 - Secure password hashing and authentication mechanisms
 - Modular backend structure for easier scalability and maintenance
 - Support for both manual and automated workflows (approval, reminders, exports)
 - Clean and intuitive UI design using Vue.js
-

8. Video Presentation

Drive Link:

https://drive.google.com/drive/folders/1W2V3XI3_S3JwktFeoNJdRexQi9j7tK38?usp=sharing